

CHAPITRE : RMI

Le but de RMI est de permettre l'appel, l'exécution et le renvoi du résultat d'une méthode exécutée dans une machine virtuelle différente de celle de l'objet l'appelant. Cette machine virtuelle peut être sur une machine différente pourvu qu'elle soit accessible par le réseau.

La machine sur laquelle s'exécute la méthode distante est appelée serveur.

L'appel coté client d'une telle méthode est un peu plus compliqué que l'appel d'une méthode d'un objet local mais il reste simple. Il consiste à obtenir une référence sur l'objet distant puis à simplement appeler la méthode à partir de cette référence.

La technologie RMI se charge de rendre transparente la localisation de l'objet distant, son appel et le renvoi du résultat.

Les différentes étapes pour créer un objet distant et l'appeler avec RMI

Le développement coté serveur se compose de :

- La définition d'une interface qui contient les méthodes qui peuvent être appelées à distance
- L'écriture d'une classe qui implémente cette interface
- L'écriture d'une classe quiinstanciera l'objet et l'enregistrera en lui affectant un nom dans le registre de noms RMI (RMI Registry)

Le développement côté client se compose de :

- L'obtention d'une référence sur l'objet distant à partir de son nom
- L'appel à la méthode à partir de cette référence

En fait, elle utilise deux classes particulières, le stub et le skeleton, qui doivent être générées avec l'outil `rmic` fourni avec le JDK.

Le stub est une classe qui se situe côté client et le skeleton est son homologue coté serveur. Ces deux classes se chargent d'assurer tous les mécanismes d'appel, de communication, d'exécution, de renvoi et de réception du résultat.

Il faut générer les classes stub et skeleton en exécutant le programme `rmic` avec le fichier source de l'objet distant.

Le développement coté serveur

La définition d'une interface qui contient les méthodes de l'objet distant

L'interface à définir doit hériter de l'interface `java.rmi.Remote`. Cette interface ne contient aucune méthode mais indique simplement que l'interface peut être appelée à distance.

L'interface doit contenir toutes les méthodes qui seront susceptibles d'être appelées à distance.

La communication entre le client et le serveur lors de l'invocation de la méthode distante peut échouer pour diverses raisons telles qu'un crash du serveur, une rupture de la liaison, etc ...

Ainsi chaque méthode appelée à distance doit déclarer qu'elle est en mesure de lever l'exception `java.rmi.RemoteException`

Exemple :

```
public interface Information extends Remote {  
  
    public String getInformation() throws RemoteException;  
  
}
```

L'écriture d'une classe qui implémente cette interface

Cette classe correspond à l'objet distant. Elle doit donc implémenter l'interface définie et contenir le code nécessaire.

Cette classe doit obligatoirement hériter de la classe `UnicastRemoteObject` qui contient les différents traitements élémentaires pour un objet distant dont l'appel par le stub du client est unique.

```
public class InformationImpl extends UnicastRemoteObject implements Information {  
  
    private static final long serialVersionUID = 2674880711467464646L;  
  
    protected InformationImpl() throws RemoteException {  
        super();  
    }  
  
    public String getInformation() throws RemoteException {  
        System.out.println("Invocation de la méthode getInformation()");  
        return "bonjour";  
    }  
}
```

NB : Comme indiqué dans l'interface, toutes les méthodes distantes, mais aussi le constructeur de la classe, doivent indiquer qu'elles peuvent lever l'exception `RemoteException`. Ainsi, même si le constructeur ne contient pas de code il doit être redéfini pour inhiber la génération du constructeur par défaut qui ne lève pas cette exception.

L'écriture d'une classe pour instancier l'objet et l'enregistrer dans le registre

La marche à suivre contient trois étapes :

- la mise en place d'un security manager dédié qui est facultative
- l'instanciation d'un objet de la classe distante
- l'enregistrement de la classe dans le registre de noms RMI

```
try {  
  
    LocateRegistry.createRegistry(1099);  
  
    InformationImpl informationImpl = new InformationImpl();  
    String url = "rmi://" + InetAddress.getLocalHost().getHostAddress() + " :Port/TestRMI";  
    System.out.println("Enregistrement de l'objet avec l'url : " + url);  
  
    Naming.rebind(url, informationImpl);  
  
    System.out.println("Serveur lancé");  
} catch (RemoteException e) {  
    e.printStackTrace();  
} catch (MalformedURLException e) {  
    e.printStackTrace();  
} catch (UnknownHostException e) {  
    e.printStackTrace();  
}
```

Le développement coté client

L'obtention d'une référence sur l'objet distant à partir de son nom

Pour obtenir une référence sur l'objet distant à partir de son nom, il faut utiliser la méthode statique `lookup()` de la classe `Naming`.

Cette méthode attend en paramètre une URL indiquant le nom qui référence l'objet distant. Cette URL est composée de plusieurs éléments : le préfix `rmi://`, le nom du serveur (`hostname`) et le nom de l'objet tel qu'il a été enregistré dans le registre précédé d'un slash.

Il est préférable de prévoir le nom du serveur sous forme de paramètres de l'application ou de l'applet pour plus de souplesse.

La méthode `lookup()` va rechercher l'objet dans le registre du serveur et retourner un objet stub. L'objet retourné est de la classe `Remote` (cette classe est la classe mère de tous les objets distants).

Si le nom fourni dans l'URL n'est pas référencé dans le registre, la méthode lève l'exception `NotBoundException`.

```
try {
    Remote r = Naming.lookup("rmi://10.0.0.13 :port/TestRMI");
} catch (Exception e) {
    e.printStackTrace();
}
```

L'appel de la méthode à partir de la référence sur l'objet distant

L'objet retourné étant de type `Remote`, il faut réaliser un cast vers l'interface qui définit les méthodes de l'objet distant. Pour plus de sécurité, on vérifie que l'objet retourné est bien une instance de cette interface.

Un fois le cast réalisé, il suffit simplement d'appeler la méthode.

```
try {
    Remote r = Naming.lookup("rmi://10.0.0.13/TestRMI");
    System.out.println(r);
    if (r instanceof Information) {
        String s = ((Information) r).getInformation();
        System.out.println("chaîne renvoyée = " + s);
    }
} catch (MalformedURLException e) {
    e.printStackTrace();
} catch (RemoteException e) {
    e.printStackTrace();
} catch (NotBoundException e) {
    e.printStackTrace();
}
```